

This document describes how to set up a dual-boot system with **Windows + Ubuntu 24.04 LTS** on a single disk, and how to install **ROS 2 Jazzy** and the Python environment needed for the myCobot labs.

It is written for the RST robotics lab and assumes that you already have Windows installed on your machine and want to **add** Ubuntu without deleting Windows.

### **Important: placeholders**

In this guide we use the following placeholders:

- **<MAIN\_WINDOWS\_DISK>**: the disk that currently contains your Windows installation (C:), the EFI System Partition, and the Windows recovery partition. In the Ubuntu installer this may appear as, e.g., `/dev/nvme0n1` (NVMe SSD) or `/dev/sda` (SATA SSD/HDD).
- **<UBUNTU\_FREE\_SPACE>**: the unallocated space you create in Windows for Ubuntu (e.g. 80–150 GB).
- **<EFI\_PARTITION>**: the existing *EFI System Partition* (FAT32, usually 100–300 MB) that currently holds the Windows boot loader.

Always double-check disk sizes and labels before modifying anything.

## 1. Prepare free space in Windows

- 1) Open the Windows disk management:
  - Press **Win + X** and choose *Disk Management*.
- 2) Identify <MAIN\_WINDOWS\_DISK>, i.e. the disk that contains:
  - the **C:** partition (Windows),
  - an *EFI System Partition* (FAT32, 100–300 MB),
  - one or more recovery partitions.
- 3) Right-click on the Windows partition **C:** and choose *Shrink Volume*.
  - Shrink by the size you want to allocate to Ubuntu (recommended: at least 80–150 GB for the lab, more if you plan to install additional software).
- 4) After shrinking, you should see **Unallocated** space on <MAIN\_WINDOWS\_DISK>. This will be <UBUNTU\_FREE\_SPACE>.

## 2. Create Ubuntu USB (UEFI)

- 1) Download the Ubuntu 24.04 LTS ISO from the official website.
- 2) Use a tool like **Rufus** on Windows to create a bootable USB:
  - Partition scheme: **GPT**
  - Target system: **UEFI (non-CSM)**
  - File system: **FAT32**
- 3) Write the ISO to the USB stick and safely eject it.

### Recommended way to boot from USB (Advanced startup)

- 1) Open *Settings* → *System* → *Recovery*.
- 2) Under *Advanced startup*, click *Restart now*.
- 3) In the blue menu, choose *Use a device*.
- 4) Select the entry corresponding to your USB drive (e.g. *UEFI: USB*).

The system should reboot from the Ubuntu USB stick.

### 3. Ubuntu installation (manual partitioning)

**Goal:** Install Ubuntu into `<UBUNTU_FREE_SPACE>` on `<MAIN_WINDOWS_DISK>` and reuse the existing `<EFI_PARTITION>` without formatting it.

- 1) When the Ubuntu menu appears, choose *Try or Install Ubuntu* (the first option).
- 2) Follow the initial steps (language, keyboard, network) until you reach the *Installation type* screen.
- 3) Choose **Manual installation** or **Something else** (do not choose *Erase disk and install Ubuntu*).
- 4) In the partitioning screen:
  - Locate `<MAIN_WINDOWS_DISK>`, e.g. `/dev/nvme0n1` or `/dev/sda`.
  - Verify it contains:
    - an EFI System Partition (FAT32) → `<EFI_PARTITION>`,
    - the Windows partition (NTFS),
    - possibly recovery partitions,
    - **free space**  $\approx$  your chosen Ubuntu size.
- 5) **Create the Ubuntu root partition** in `<UBUNTU_FREE_SPACE>`:
  - a) Select the *free space* on `<MAIN_WINDOWS_DISK>`.
  - b) Click *Add* and create:
    - Type: **Primary**,
    - File system: **Ext4**,
    - Mount point: `/`,
    - Size: use the full free space (or leave 8–16 GB unallocated if you plan to create a separate swap partition later).
- 6) **Reuse the existing EFI partition (do not format!)**
  - a) Select `<EFI_PARTITION>` (FAT32, EFI System Partition).
  - b) Set:
    - Use as: **EFI System Partition**,
    - Mount point: `/boot/efi`,
    - Format: **unchecked** (must not be formatted).
  - c) If the installer insists on formatting this partition, abort the installation and ask a tutor.

**7) Boot loader location**

- Set *Device for boot loader installation* to `<MAIN_WINDOWS_DISK>` (the whole disk, e.g. `/dev/nvme0n1`), not a specific partition and not the USB drive.

**8) Continue with the installation.**

- Choose your username, password, etc.

After installation completes, the system will reboot. Remove the USB stick when prompted. On the next boot you should see the GRUB menu with entries for both Ubuntu and Windows.

## 4. First steps in Ubuntu

Once you have booted into Ubuntu 24.04 for the first time:

1) Open a terminal (e.g. Ctrl + Alt + T).

2) Update the system:

```
sudo apt update  
sudo apt upgrade -y
```

3) Optionally install some basic tools (e.g. build essentials, git):

```
sudo apt install -y build-essential git
```

## 5. Python virtual environment for the myCobot labs

For the robotics labs we use a dedicated Python virtual environment. All Python packages required by the lab will be installed in this environment.

- 1) Install Python and venv support (usually already present):

```
sudo apt install -y python3 python3-pip python3-venv python3-dev python3-full
```

- 2) Create a folder that will hold all your virtual environments:

```
mkdir -p ~/venvs
```

- 3) Create the virtual environment for the myCobot labs:

```
python3 -m venv ~/venvs/mycobot
```

- 4) Activate the virtual environment:

```
source ~/venvs/mycobot/bin/activate
```

- 5) Upgrade packaging tools **inside** the virtual environment:

```
python -m pip install -U pip setuptools wheel
```

- 6) (Optional) Install additional common tools into the venv:

```
python -m pip install -U colcon-common-extensions vcstool rosdep
```

- 7) When you are done, deactivate the environment:

```
deactivate
```

- 8) To use it again later (since we would be running the ROS2 commands from the venv), reactivate it:

```
source ~/venvs/mycobot/bin/activate
```

## 6. Install ROS 2 Jazzy

We now install **ROS 2 Jazzy** from the official packages.

- 1) Make sure the system is up to date:

```
sudo apt update
sudo apt upgrade -y
```

- 2) Configure locale (required by some ROS tools):

```
sudo apt install -y locales
sudo locale-gen en_US en_US.UTF-8
sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8
export LANG=en_US.UTF-8
```

- 3) Enable the universe repository:

```
sudo apt install -y software-properties-common
sudo add-apt-repository universe -y
```

- 4) Add the ROS 2 apt repository and key:

```
sudo apt install -y curl ca-certificates gnupg
sudo mkdir -p /etc/apt/keyrings

# Download the ROS key to a temporary file
curl -sSL https://raw.githubusercontent.com/ros/rosdistro/master/ros.key -o /tmp/ros.key

# Convert it to a keyring
sudo gpg --dearmor -o /etc/apt/keyrings/ros-archive-keyring.gpg /tmp/ros.key

ARCH=$(dpkg --print-architecture)
CODENAME=$(cat /etc/os-release | grep SUBUNTU_CODENAME | cut -d= -f2)
ROS_KEYRING=/etc/apt/keyrings/ros-archive-keyring.gpg
ROS_LIST=/etc/apt/sources.list.d/ros2.list
ROS_URL=http://packages.ros.org/ros2/ubuntu

echo "deb [arch=${ARCH} signed-by=${ROS_KEYRING}] ${ROS_URL} ${CODENAME} main" | sudo tee ${ROS_LIST} > /dev/null

sudo apt update
```

- 5) Install ROS 2 Jazzy (desktop variant):

```
sudo apt install -y ros-jazzy-desktop
```

- 6) Source ROS automatically for new terminals:

```
echo "source /opt/ros/jazzy/setup.bash" >> ~/.bashrc
source /opt/ros/jazzy/setup.bash
```

- 7) Install additional ROS 2 tools:

```
sudo apt install -y python3-rosdep python3-colcon-common-extensions
sudo rosdep init
rosdep update
```

## 7. Quick ROS 2 test

1) In a terminal:

```
ros2 --help  
ros2 run demo_nodes_cpp talker
```

2) In a second terminal:

```
source /opt/ros/jazzy/setup.bash  
ros2 run demo_nodes_cpp listener
```

3) If the listener prints messages from the talker, your ROS 2 installation is working.

You now have:

- a dual-boot system (Windows + Ubuntu 24.04),
- a dedicated Python virtual environment `/venvs/mycobot`, and
- a working installation of ROS 2 Jazzy,

which are the prerequisites for the myCobot robotics labs.